

חלק א – שאלות הבנה וניתוח על המעבדה (החלק הרטוב)

חלק זה בודק שאכן הבנתם את מה שמימשתם ואת מה שהניסויים בנוטבוק מראים. ענו בקצרה ולעניין. כאשר שאלה מתייחסת לתוצאה מספרית, הסתמכו על הטבלאות והגרפים שהפקתם בנוטבוק (חלקים 1-6).

שימו לב: לכל סעיף מוקצה שדה תשובה משלו, וגודלו משקף את אורך התשובה המצופה. כתבו תשובה קצרה וממוקדת בתוך השדה המתאים; אין צורך להרחיב מעבר לכך.

1.א. הבנת המימוש

שאלה 1: הסדר predict-then-update

(3 נק')

- (1) הסבירו במשפט אחד מדוע חובה לחשב את החיזוי (predict) לפני עדכון רגיסטר ההיסטוריה ומוני ה-PHT: איזה מידע, שאינו זמין לחזאי אמיתי בזמן החיזוי, כבר "מוטמע" במצב המעודכן?
- (2) נניח שבתוך update מעדכנים קודם את רגיסטר ההיסטוריה ורק אחר כך מחשבים את אינדקס ה-PHT. מאיזו היסטוריה (הישנה או החדשה) ייגזר האינדקס במקרה זה, ואיזה מונה יעודכן בטעות?

תשובה:

(1) במצב המעודכן מוטמע התוצאה האמיתית של הbranch. בחזאי אמיתי, בזמן הprediction הוא לא יודע מה תהיה התוצאה האמיתית (זה כמובן כל הפואנטה של הרכיב). לכן, אם קודם נעשה update ורק אז predict, אז בעצם נדע כבר את התוצאה האמיתית וכך נחזה, בשונה מחזאי אמיתי.

(2) האינדקס ייגזר מההיסטוריה: ☐ הישנה ☒ החדשה. המונה שיעודכן בטעות:

המונה של הקפיצה הבאה (כלומר ההיסטוריה כוללת את הקפיצה הנוכחית)

(3 נק')

שאלה 2: מדוע מונה בן 2 ביטים?

- (1) כמה חיזויים שגויים רצופים נדרשים כדי שמונה רווי בן 2 ביטים יהפוך את כיוון החיזוי שלו (למשל מ-T ל-N)?
- (2) על דפוס של לולאה ארוכה ($T \dots T \dots T$ וחוזר חלילה), כמה חיזויים שגויים בכל איטרציה מבצע במצב יציב מונה בן ביט אחד, וכמה מבצע מונה בן 2 ביטים? הסבירו במשפט אחד מה קורה אצל כל אחד מהם סביב הקפיצה הבודדת N שבסוף האיטרציה.

תשובה:

(1) מספר חיזויים שגויים רצופים:

(2) מונה 1 ביט: טעויות לאיטרציה; מונה 2 ביטים: טעויות לאיטרציה.

מונה עם ביט 1: בכל איטרציה כשיגיע הbranch האחרון הוא יחשוב שצריך לקפוץ ולכן יטעה. לאחר מכן הוא יעדכן את המידע וכעת יחזה שלא נקפוץ, כיוון שנעבור לאיטרציה הבאה, הbranch הראשון שם גם יטעה. כלומר בכל איטרציה נטעה פעמיים, מלבד האיטרציה הראשונה בה נטעה רק בסוף (תלוי בברירת המחדל).

מונה עם 2 ביטים: כיוון שזו לולאה ארוכה, המונה יהיה רווי ולכן יטעה הbranch האחרון, אבל עדיין ישאר במצב של taken ולכן זו תהיה הטעות היחידה שלו.

(3 נק')

שאלה 3: מסכה ו-XOR ב-get_pht_index

- (1) ב-PHT יש $2^{history_size}$ כניסות. מהו טווח האינדקסים החוקי, ומה עלול לקרות בגישה לטבלה אילולא המסכה (& mask)?
- (2) כש-share = True מבצעים XOR בין ההיסטוריה לבין הביטים התחתונים של כתובת הקפיצה. במשפט אחד – מה המטרה של ה-XOR הזה?

תשובה:

(1) טווח האינדקסים החוקי: $2^{history_size}$. בלי המסכה:
 היינו ניגשים לכתובות בטבלה שאינן נמצאות בטווח האינדקסים של הטבלה – כתובות לא חוקיות.

(2)
 בלי XOR, פקודות שונות יכולות לגשת לאותו המונה ולדרוס את המידע שבו. בעזרת XOR ניתן לבצע הפרדה טובה יותר בין פקודות שונות והיסטוריות שונות. מעין hashing של ההיסטוריה, ושל זהות הbranch.

(3 נק')

שאלה 4: טיפול ב-tag בטבלה הלוקאלית

- בטבלת ההיסטוריה הלוקאלית, כל כניסה שומרת ב-bit tag, valid, ורגיסטר היסטוריה bhr. שתי כתובות שונות יכולות להיות ממופות לאותה כניסה (tag miss).
- הערה:** אין מוסכמה אוניברסלית אחת לגבי כיוון החיזוי לאחר tag miss – מעבדים שונים נוהגים אחרת. בתרגיל זה (ובמימוש בחלק הרטוב) ברירת המחדל היא Not Taken.
- (1) במסלול החיזוי (predict): מה יהיה כיוון החיזוי של הקפיצה במקרה זה, והאם מצב החזאי משתנה?
- (2) במסלול העדכון (update) באותו מצב: אילו שדות של הכניסה מאותחלים מחדש, ולאילו ערכים?
- (3) ציינו יתרון אחד של ברירת המחדל Not Taken עבור קפיצה שהחזאי עדיין לא למד.

תשובה:

(1) החיזוי: ☒ T ; ☐ N; מצב החזאי משתנה: ☐ כן ☒ לא.

(2)
 זה מאתחל את ההיסטוריה להיות 0
 את tag להיות החדש.
 valid = true

(3)
 כאשר מניחים branch לא נלקח, המעבד ממשיך להביא את הפקודות הרציפות, שבדרך כלל נמצאות קרוב זו לזו בזיכרון ולכן יש סיכוי גבוה שהן כבר נמצאות בזיכרון המטמון. אם היינו מניחים שהbranch כן נלקח והתחזית הייתה שגויה, המעבד היה מבזבז זמן בהבאת פקודות מזיכרון פיזי שלא לצורך. כיוון שזו ברירת מחדל, עדיפה לנו האפשרות בה יש פחות בזבז משאבים.

(3 נק')

שאלה 5: חישוב אינדקס ו-tag

נתון $history_table_size = 8$ וכתובת קפיצה $0x1A4$.

הערה: השתמשו בכתובת כפי שהיא, ללא הסרת ביטי יישור – ה-index נלקח מהביטים התחתונים של הכתובת וה-tag מהביטים העליונים שמעליהם.

(1) מהו $get_local_index(0x1A4, 8)$? הראו את חישוב הביטים.

(2) מהו $get_local_tag(0x1A4, 8)$? הראו את חישוב הביטים.

(3) כמה ביטים תופס ה-index וכמה תופס ה-tag עבור $address_bits = 32$?

תשובה:

חישוב הביטים:

גודל הטבלה 8 ולכן נדרשים 3 ביטים (התחתונים) עבור האינדקס. הגדרנו כי התגית תהיה 29 ביטים העליונים.
 $0x1a4 = 000110100100$, $index = 0x1a4 \text{ AND } 111 \implies 100$
 tag:
 shift 000110100100 right by $\log(8) \implies tag = 000110100$

index = tag =
 index bits = tag bits =

2. ניתוח תוצאות הניסויים

השאלות הבאות מבוססות על הטבלאות והגרפים שהפקתם בנוטבוק (במיוחד טבלת ההשוואה בחלק 6 וגרפי ה-PHT accesses/timeline).

(4 נק')

שאלה 6: החזאי הטוב ביותר

איזה חזאי השיג את הדיוק הממוצע הגבוה ביותר על פני כל ה-traces? ציינו את שמו ואת הדיוק הממוצע שלו מתוך הטבלה, ונמקו במשפט אחד (על סמך ה-traces שבהם הוא בולט לטובה) מדוע דווקא הוא.

תשובה:

החזאי: ; דיוק ממוצע: %

פעולת הxor של Lshare מפצלת את הפקודות למקומות שונים גם כאשר ההיסטוריה זהה ובכך מונעת קונפליקטים.

(4 נק')

שאלה 7: מתי היסטוריה לוקאלית עוזרת?

באיזה trace ההיסטוריה הלוקאלית עזרה בבירור? ציינו את שם ה-trace, תארו בקצרה את הדפוס של כל אחת מפקודות הקפיצה שבו, והסבירו במשפט אחד מדוע דווקא היסטוריה נפרדת לכל פקודת קפיצה מתאימה לדפוסים אלה.

תשובה:

trace:

פקודה אחת עם תבנית TTTN ופקודה שנייה TN
לכל branch יש תבנית מאוד פשוטה שקל להבין, אך כיוון שהפקודות בשתי וערב, היסטוריה גלובלית לא תזהה את התבנית בקלות. לכן כששומרים לכל branch היסטוריה פרטית ניתן לזהות בקלות את התבנית.

(4 נק')

שאלה 8: מתי היסטוריה גלובלית עוזרת?

באיזה trace ההיסטוריה הגלובלית עזרה בבירור? ציינו את שם ה-trace, וכתבו במדויק היכן שמורה התוצאה האחרונה של 0x100 בזמן החיזוי של 0x104 – ומדוע חזאי עם היסטוריה לוקאלית אינו רואה אותה.

תשובה:

trace:

בזמן החיזוי של 0x104, התוצאה האחרונה של שמורה בביט האחרון (החדש ביותר) של ההיסטוריה כיוון שהפקודות הן בשתי וערב. חזאי עם היסטוריה לוקלית רואה רק את ההיסטוריה של הbranch המסוים (כיוון שכל היסטוריה נשמרת בנפרד לכל פקודה), ולכן החזאי עבור 0x104 לא ידע לבדוק את ההיסטוריה של 0x100.

(4 נק')

שאלה 9: דוגמה ל-aliasing הרסני

הביאו דוגמה קונקרטית אחת ל-aliasing הרסני מתוך הגרפים או הטבלאות שהפקתם: ציינו את שם ה-trace ואת כתובות שתי פקודות הקפיצה המתנגשות, והסבירו במשפט אחד מדוע ההתנגשות פוגעת בדיוק. בחרו מקרה שונה מזה של aliasing_bad הנדון בשאלה הבאה – למשל trace שבו gshare כן מצליח להפריד בין שתי פקודות הקפיצה.

תשובה:

trace: ; כתובות: ,

כשקוראים ל-100x בשורה 13 עם היסטוריה TTTT המונה שיעודכן הוא 1111 (המונה יעלה) ואז בשורה 18 כשקוראים ל-104x עם היסטוריה TNTT המונה שיעודכן הוא גם 1111 (הפעם המונה ירד).

(4 נק')

שאלה 10: gshare מול global-global ב-aliasing_bad

ב-aliasing_bad.trace, gshare משיג בדיוק את אותו דיוק כמו global-global. כתבו את ארבעת הביטים התחתונים של כל אחת מהכתובות 0x100 ו-0x200, והסבירו במשפט אחד מדוע ה-XOR עם הכתובת אינו משנה את אינדקס ה-PHT במקרה זה.

תשובה:

0x100 → 0000 0x200 → 0000

על פי המימוש שלנו, בהיסטוריה משותפת נעשה xor עם הביטים התחתונים של הכתובת. מספריות, לעשות XOR עם 0 לא משנה את הביט. לכן כשעושים xor של ההיסטוריה עם 4 הביטים התחתונים של הכתובת, נשארים עם אותו אינדקס.

(5 נק')

שאלה 11: warm-up ב-always_taken

על always_taken.trace (פקודת קפיצה יחידה שתמיד Taken) כל החזאים טועים בדיוק 5 פעמים (75% דיוק) למרות שהדפוס פשוט מאוד.

- (1) מדוע יש בכלל טעויות, אם הקפיצה תמיד Taken?
- (2) הסבירו מדוע מספר הטעויות הוא בדיוק 5 עבור global-global עם history_size = 4: רשמו את כל ערכי ה-GHR השונים שהרגיסטר עובר דרכם עד שהוא מתייצב, וכתבו במשפט אחד מדוע כל ערך כזה גורר בדיוק טעות אחת.

תשובה:

(1) כל מונה מאותחל ל-WNT ולכן בניהוש הראשון תמיד ננחש not taken למרות שבפועל זה יהיה taken

(2) ערכי ה-GHR (בבינארית): 0000 0001 0011 0111 1111

לכל היסטוריה אפשרית, המונה מאותחל ל-WEAKLY NOT TAKEN ולכן בפעם הראשונה ננחש not taken ונטעה, לאחר מכן המונה יעודכן ל-TAKEN ונצדוק בכל חיזוי עתידי.

(5 נק')

שאלה 12: דיוק מול אחסון

האם החזאי המדויק ביותר הוא תמיד הבחירה הנכונה? בחרו מהטבלה מקרה אחד שבו חזאי יקר בהרבה משיג שיפור דיוק קטן בלבד (או לא משיג שיפור כלל) לעומת חזאי זול: מלאו את הנתונים של שני החזאים, וסכמו במשפט אחד את המסקנה.

תשובה:

חזאי זול: GSHARE; אחסון: 36 ביטים; דיוק: 77.75%

חזאי יקר: local-local; אחסון: 1040 ביטים; דיוק: 74.97%

איזה חזאי עדיף תלוי במה תבנית פקודות ה-branch, חזאים שונים מתאימים למצבים שונים, ואין קשר ישיר בין המחיר לבין הדיוק.

חלק ב – שאלות יבשות (עיוניות)

חמש שאלות עיוניות. כתבו את תשובותיכם בתוך תיבות התשובה, בשדה המסומן של כל סעיף. גודל השדה משקף את אורך התשובה המצופה.

שאלה 13: סימולציה ידנית של מונה רווי בן 2 ביטים (5 נק')

נתון מונה רווי בן 2 ביטים שמאותחל ל-WNT. נתון הרצף הבא של תוצאות אמיתיות:

T T N T N N T

הנחיות מילוי הטבלה. בכל צעד:

- מצב לפני – ערך המונה כעת (לפני העדכון).
- חיזוי – T או N לפי המצב לפני.
- תוצאה – התוצאה האמיתית הנתונה.
- מצב אחרי – ערך המונה לאחר העדכון.
- נכון? – סמנו V אם החיזוי תאם את התוצאה, אחרת X.

צעד	מצב לפני	חיזוי	תוצאה	מצב אחרי	נכון?
1	01 (WNT)	N	T	10	X
2	10	T	T	11	V
3	11	T	N	10	X
4	10	T	T	11	V
5	11	T	N	10	X
6	10	T	N	01	X
7	01	N	T	10	X

(ב) כמה חיזויים נכונים התקבלו בסך הכול? מהו אחוז הדיוק על רצף זה?

תשובה:

חיזויים נכונים: 2 מתוך 7; דיוק: 28.57%

(5 נק')

שאלה 14: מדוע היסטוריה עוזרת?

נתון רצף תוצאות של פקודת קפיצה עם דפוס מחזורי:

T T T N T T T N T T T N ...

(1) באיזה צעד בדפוס טועה, במצב יציב, מונה 2-bit ללא היסטוריה – ומדוע המונה אינו "לומד" את הצעד הזה לעולם?

(2) מהו אורך ההיסטוריה המינימלי הנדרש כדי לחזות את הדפוס במדויק (לאחר חימום)?

(3) עבור אורך זה, התייחסו רק לערכי ה-BHR שמופיעים במצב יציב (לאחר החימום), כאשר הביט הפחות-משמעותי (LSB) מייצג את התוצאה האחרונה. אילו מהם חוזים T ואילו חוזים N? ציינו במפורש את ערכי הביטים.

תשובה:

(1)

הוא תמיד יטעה בצעד של NOT TAKEN. כיוון ש-3 TAKEN ברצף תמיד יגרמו למונה להיות תמיד ב-STRONGLY TAKEN, ולכן נחזה בשלב הבא TAKEN ונטעה. בכל שאר הצעדים נחזה TAKEN ובאמת יהיה TAKEN.

(2) אורך היסטוריה מינימלי:

(3) BHR שחוזים N: ; BHR שחוזים T:

(5 נק')

שאלה 15: היסטוריה לוקאלית מול גלובלית

נתון התרחיש הבא:

- פקודת קפיצה A בעלת דפוס לא סדיר (תלוי-נתונים), שאינו ניתן לחיזוי מתוך ההיסטוריה של A עצמה.
 - פקודת קפיצה B שתוצאתה תמיד שווה ל-תוצאה האחרונה של A.
 - הקריאות מבוצעות לסירוגין: A, B, A, B, A, B, ...
- (1) איזה חזאי צפוי לחזות טוב יותר את פקודת הקפיצה B: היסטוריה לוקאלית או היסטוריה גלובלית? נמקו במשפט אחד.
- (2) הסבירו במשפט אחד מדוע היסטוריה גלובלית יכולה ללכוד קורלציה בין פקודות קפיצה שונות.
- (3) תארו תרחיש קונקרטי שבו דווקא היסטוריה לוקאלית עדיפה על גלובלית: ציינו שתי פקודות קפיצה ואת דפוס התוצאות של כל אחת מהן (למשל A: T, T, N, T, T, N, ...), והסבירו במשפט אחד מדוע הרגיסטר הגלובלי המשותף נכשל שם.

תשובה:

(1) החזאי העדיף ל-B: ☐ לוקאלית ☒ גלובלית. נימוק:

כיוון ש B תלויה לחלוטין ב A, היסטוריה גלובלית (אפילו באורך 1) תכיל את תוצאת A ולכן בעזרתה ניתן לחזות בדיוק את תוצאת B.

(2)

היסטוריה גלובלית מכילה תוצאות של מספר פקודות BRANCH שונות, ולכן יכולה לזהות תבנית המתקיימת בקשר שבין תוצאותיה.

(3) פקודת קפיצה A: ; פקודת קפיצה B:

התרחיש הוא כשקודם A ואז B, על פי המחזוריות שהוגדרה וכאשר אורך ההיסטוריה הוא 2. אזי במצב לוקלי, מספיקים 2 ביטים בשביל לזהות את המחזוריות של כל פקודה בנפרד ולכן במצב יציב נחזה במדויק. במצב גלובלי, נקבל כי 2 ביטים אינם מספיקים בשביל לחזות.

(5 נק')

שאלה 16: Aliasing ו-gshare

נתונות שתי פקודות קפיצה שממופות לאותו אינדקס ב-PHT הגלובלי כאשר `share = False`:

- פקודת קפיצה A – ברוב המקרים Taken.
- פקודת קפיצה B – ברוב המקרים Not Taken.

- (1) האם ההפרעה ביניהם היא בונה (constructive) או הרסנית (destructive)? נמקו במשפט אחד.
- (2) הסבירו במשפט אחד כיצד XOR עם הביטים התחתונים של כתובת הקפיצה (gshare) יכול להפריד את שתי פקודות הקפיצה לאינדקסים שונים.
- (3) הדגימו מספרית מקרה שבו ה-XOR דווקא יוצר aliasing חדש: בחרו עבור כל פקודת קפיצה ביטים תחתונים של כתובת וערך היסטוריה (4 ביטים), כך ששני הזוגות השונים ממופים לאותו אינדקס PHT.

תשובה:

(1) ההפרעה: ☐ בונה ☒ הרסנית. נימוק:

פקודה A תעלה את המונה, ופקודה B תוריד אותו. כלומר כל אחת מהן תפגע ביכולת החיזוי של הפקודה השנייה.

(2)

מכיוון שלשתי הפקודות השונות יש כתובות שונות (וכפועל יוצא, ככל הנראה גם ביטים תחתונים שונים), ביצוע פעולת XOR של הכתובות השונות הללו עם אותו ערך של היסטוריה גלובלית יניב תוצאות שונות, וכך ימפה כל פקודה לאינדקס נפרד ב-PHT.

(3)

A: addr = , hist = ; B: addr = , hist = ; index =

(5 נק')

שאלה 17: חישוב CPI

נתונים הפרמטרים הבאים של מעבד:

$$CPI_{base} = 1$$

שכיחות פקודות קפיצה = 20%

עונש misprediction = 10 מחזורים

misprediction rate: A = 10%

misprediction rate: B = 4%

חשבו את ה-CPI עבור כל אחד מהחזאים, לפי הנוסחה

$$CPI_{eff} = CPI_{base} + branch_freq \cdot misprediction_rate \cdot penalty$$

ועל סמך החישוב ענו:

(1) מהו CPI_{eff} של חזאי A?(2) מהו CPI_{eff} של חזאי B?

(3) מהו השיפור היחסי ב-CPI של חזאי B ביחס לחזאי A (באחוזים)?

(4) הציעו שיקול אחד שיכול בכל זאת לגרום למתכנני מעבד לבחור בחזאי A (הפשוט יותר) על פני חזאי B.

תשובה:

$$CPI_{eff}(A) = 1.2 \quad CPI_{eff}(B) = 1.08 \quad improvement = 10\%$$

(4)

יכול להיות שמעבד A רץ בתדירות גבוהה יותר, ולכן למרות שצריך יותר מחזורי שעון לכל פקודה, הוא בכל זאת יהיה מהיר יותר.

בהצלחה!